

ESCOLA SUPERIOR NÁUTICA INFANTE D. HENRIQUE

Computação Móvel

Tutorial 2 — Secção 2

Cool Weather Application

Relatório de Desenvolvimento

Unidade Curricular:

Computação Móvel (CM)

Aluno

Diogo Miguel de Oliveira Martins

Docentes:

Pedro Fazenda | Carlos Gonçalves

1. Resumo Executivo

O presente relatório documenta o desenvolvimento da Cool Weather App no âmbito do Tutorial 2 de Computação Móvel. A aplicação foi implementada em Kotlin para Android, consumindo a API pública Open-Meteo para apresentar dados meteorológicos em tempo real. O projeto encontra-se numa fase avançada de desenvolvimento, com a maioria das funcionalidades core já implementadas e funcionais.

Componente	Estado	Observação
Chamada à API Open-Meteo	✓ Completo	Gson + URL.openStream() em thread separada
Atualização da UI	✓ Completo	runOnUiThread com tratamento de erro
Layout Portrait (telemóvel)	✓ Completo	ScrollView + ConstraintLayout
Layout Landscape (telemóvel)	✓ Completo	Guideline 35% + ImageView lateral
Temas Day/Night	⚠ Parcial	Backgrounds iguais no portrait e landscape
Ícone meteorológico (WMO)	✓ Completo	getIdentifier() dinâmico com fallback
Internacionalização PT/EN	✓ Completo	strings.xml + values-pt/strings.xml
Permissão INTERNET	✓ Completo	AndroidManifest.xml configurado
Ícone personalizado	✓ Completo	ic_launcher-playstore.png presente
Layout Tablet	✗ Em falta	Pasta layout-sw600dp não existe
Fetch inicial no onCreate	✗ Em falta	App arranca sem dados meteorológicos
Background Land diferente	⚠ Parcial	Day.Land usa mesmo drawable que Portrait

2. Estrutura do Projeto

O projeto segue a estrutura padrão de um projeto Android Studio com os seguintes ficheiros fonte relevantes:

Ficheiro	Descrição
MainActivity.kt	Activity principal com toda a lógica de UI e API
WeatherData.kt	Data classes, enum WMO e getWeatherCodeMap()
res/layout/activity_main.xml	Layout portrait para telemóvel
res/layout-land/activity_main.xml	Layout landscape para telemóvel
res/values/themes.xml	4 temas: Day, Day.Land, Night, Night.Land
res/values-night/themes.xml	Override de tema para modo escuro do sistema
res/values/strings.xml	Strings em inglês (locale padrão)
res/values-pt/strings.xml	Strings em português
res/values/colors.xml	Paleta de cores (mínima)
res/drawable/verticando.jpg	Background diurno
res/drawable/noitando.png	Background noturno
res/drawable/*.xml	28 ícones meteorológicos (Vector Assets)
AndroidManifest.xml	Permissão INTERNET + Activity declarada

3. Componentes Implementados

3.1 Modelo de Dados — WeatherData.kt

O ficheiro WeatherData.kt implementa corretamente toda a estrutura de dados necessária para deserializar a resposta da API Open-Meteo:

```
data class WeatherData(latitude, longitude, timezone, current_weather,
    hourly)
data class CurrentWeather(temperature, windspeed, winddirection, weathercode,
    time)
data class Hourly(time, temperature_2m, weathercode, pressure_msl)
```

O enum `WMO_WeatherCode` mapeia todos os 28 códigos WMO relevantes (0 a 99) a nomes de drawables, com a função `getWeatherCodeMap()` a devolver um `HashMap<Int, WMO_WeatherCode>` para consulta eficiente.

3.2 Integração com a API — MainActivity.kt

A integração com a API Open-Meteo está implementada de forma robusta e segue boas práticas Android:

WeatherAPI_Call()

Constrói o URL dinamicamente com latitude, longitude e as variáveis hourly necessárias. A deserialização é feita com Gson diretamente do InputStream, incluindo tratamento de exceções com try/catch que evita crashes em caso de falha de rede.

```
val reqString = buildString {
    append("https://api.open-meteo.com/v1/forecast?")
    append("latitude=${lat}&longitude=${long}&")
    append("current_weather=true&timezone=auto&")
    append("hourly=temperature_2m,weathercode,pressure_msl,windspeed_10m")
}
url.openStream().use {
    Gson().fromJson(InputStreamReader(it, "UTF-8"), WeatherData::class.java)
}
```

Nota positiva: o parâmetro `timezone=auto` foi adicionado, o que é uma melhoria face ao tutorial base, garantindo que as horas retornadas estão no fuso correto da localização.

fetchWeatherData() + updateUI()

O padrão Thread + runOnUiThread está corretamente implementado: o pedido de rede corre numa thread de background e a atualização da UI é feita na UI thread. Há ainda tratamento explícito do caso em que a API retorna null (erro de rede), apresentando um Toast de erro ao utilizador.

```
private fun fetchWeatherData(lat: Float, long: Float) {
    Thread {
        val weather = WeatherAPI_Call(lat, long)
        if (weather != null) {
            runOnUiThread { updateUI(weather) }
        } else {
            runOnUiThread { Toast.makeText(...).show() }
        }
    }.start()
}
```

updateUI()

Atualiza corretamente todos os TextViews com os valores da API: temperatura, velocidade do vento, direção do vento, tempo e pressão ao nível do mar (índice 12 das listas hourly, correspondente às 12h do dia). A seleção do ícone meteorológico é feita dinamicamente:

```
val wImage = when (wCode) {
    CLEAR_SKY, MAINLY_CLEAR, PARTLY_CLOUDY ->
        if (day) wCode.image + "day" else wCode.image + "night"
    else -> wCode?.image ?: "clear_day" // fallback
}
val resID = resources.getIdentifier(wImage, "drawable", packageName)
if (resID != 0) imgWeather.setImageResource(resID)
else imgWeather.setImageResource(R.drawable.clear_day)
```

Ponto positivo: o fallback duplo (elvis operator + verificação de resID != 0) garante que a app nunca crasha por um drawable não encontrado.

4. Layout e Interface

4.1 Layout Portrait (activity_main.xml)

O layout portrait usa um `ScrollView` como raiz (boa prática para ecrãs pequenos) com um `ConstraintLayout` interior. Os componentes estão alinhados em dois grupos: labels à esquerda e valores à direita, usando constraints de baseline para alinhamento horizontal. O botão `UPDATE` está corretamente ancorado ao fundo com constraint `top/bottom`.

Componente	ID / Configuração
ImageView	imgWeather — 120×120dp, topo centrado
EditText (Lat)	editLat — numberDecimal numberSigned
EditText (Long)	editLong — numberDecimal numberSigned
TextView × 7	Labels: Latitude, Longitude, Sea Level Pressure, Wind Direction, Wind Speed, Temperature, Time
TextView × 5	Valores: PressureValue, WindDirections, WindSpeeds, txtTemperature, TimeValue
Button	btnUpdate — backgroundTint #9C27B0 (roxo)

4.2 Layout Landscape (layout-land/activity_main.xml)

O layout landscape usa uma `Guideline` vertical a 35% para dividir o ecrã em dois painéis: o `ImageView` do tempo ocupa o painel esquerdo (0–35%) e os campos de dados e botão `UPDATE` ocupam o painel direito (35–100%). Esta abordagem é eficaz e resulta numa interface visualmente equilibrada em modo horizontal.

4.3 Temas (themes.xml)

Os quatro temas estão definidos e aplicados corretamente em `onCreate()`:

```
when (resources.configuration.orientation) {
    ORIENTATION_PORTRAIT ->
        setTheme(if (day) R.style.Theme_Day else R.style.Theme_Night)
    else ->
        setTheme(if (day) R.style.Theme_Day_Land else
R.style.Theme_Night_Land)
}
```

Nota: atualmente os temas `Theme.Day` e `Theme.Day.Land` usam o mesmo drawable (verticando), e os temas `Night` partilham noitando. Seria mais correto ter um crop diferente da imagem para landscape.

4.4 Internacionalização

As strings estão corretamente divididas entre os dois ficheiros. O ficheiro em inglês serve de locale padrão e o português está definido em values-pt/:

String ID	Inglês	Português
<code>pressure_label</code>	Sea Level Pressure	Pressão ao Nível do Mar
<code>wind_dir_label</code>	Wind Direction	Direção do Vento
<code>wind_speed</code>	Wind Speed	Velocidade do Vento
<code>temp_label</code>	Temperature	Temperatura
<code>time_label</code>	Time	Hora
<code>btn_update</code>	UPDATE	ATUALIZAR

5. Problemas Identificados e Correções Sugeridas

5.1 Fetch Inicial Ausente no onCreate()

Gravidade: Alta. A app arranca com todos os campos a mostrar "---" porque `fetchWeatherData()` nunca é chamada no `onCreate()`. O utilizador tem de premir UPDATE manualmente para ver dados.

Correção — adicionar no final do `onCreate()`, após configurar o listener do botão:

```
// Carregar dados de Lisboa por omissão ao arrancar
fetchWeatherData(38.7f, -9.1f)
```

5.2 Layout Tablet em Falta

Gravidade: Alta. O tutorial exige explicitamente um layout para tablet (`res/layout-sw600dp/`). Sem este ficheiro, a app usa o layout de telemóvel em tablets, o que resulta numa interface inadequada.

Correção — criar a pasta `res/layout-sw600dp/` e um `activity_main.xml` adaptado, tipicamente com uma disposição diferente que aproveite o ecrã maior (ex.: imagem maior, dados em duas colunas).

5.3 Backgrounds Landscape Iguais ao Portrait

Gravidade: Média. Os temas `Theme.Day.Land` e `Theme.Night.Land` usam os mesmos drawables que os temas portrait. O tutorial pede explicitamente imagens diferentes com crop landscape (786×393 px).

Correção — adicionar um drawable de background landscape (ex.: `verticando_land.jpg` e `noitando_land.png`) e atualizar `themes.xml`:

```
<style name="Theme.Day.Land" parent="Theme.CoolWeatherApp">
    <item name="android:windowBackground">@drawable/verticando_land</item>
</style>
<style name="Theme.Night.Land" parent="Theme.CoolWeatherApp">
    <item name="android:windowBackground">@drawable/noitando_land</item>
</style>
```

5.4 Valores Iniciais dos EditText

Gravidade: Média. Os campos de latitude e longitude estão vazios no arranque. Seria mais user-friendly pré-preencher com os valores padrão de Lisboa.

Correção — adicionar no XML do EditText:

```
android:text="38.76"    <!-- para editLat -->
android:text="-9.12"    <!-- para editLong -->
```

5.5 Determinação Dinâmica de day

Gravidade: Média. A variável day está hardcoded a true. O tutorial pede que seja calculada com base nos dados de sunrise/sunset da API. Para isso, seria necessário adicionar o parâmetro daily=sunrise,sunset ao URL e comparar a hora atual da localização com esses valores.

Correção — adicionar ao URL da API:

```
append("&daily=sunrise,sunset")
```

E calcular day após receber a resposta, comparando current_weather.time com os valores de sunrise/sunset retornados.

6. Análise de Qualidade do Código

6.1 Pontos Positivos

- Tratamento robusto de erros na WeatherAPI_Call com try/catch e retorno nullable (WeatherData?)
- Fallback duplo para drawable não encontrado (elvis ?: + verificação resID != 0)
- Uso correto de buildString para construção do URL sem concatenação manual
- Toast de feedback ao utilizador quando a atualização é iniciada
- Parâmetro timezone=auto adicionado — melhoria não prevista no tutorial base
- Conversão segura de EditText para Float com toFloatOrNull() e valor padrão
- Drawables organizados corretamente na pasta drawable/
- Ícone personalizado da aplicação definido (ic_launcher-playstore.png)

6.2 Aspetos a Melhorar

- Ausência do fetch inicial no onCreate() — principal falha funcional
- Layout tablet (sw600dp) não implementado
- Variável day hardcoded, sem cálculo dinâmico de dia/noite real
- Backgrounds landscape iguais aos portrait — tema não totalmente diferenciado
- EditText sem valores iniciais (Lisbon por omissão)
- colors.xml muito minimalista — apenas black e white definidos, sem paleta própria

6.3 Conformidade com o Tutorial

Requisito do Tutorial	Estado	Peso
API Open-Meteo integrada com Gson	✓	Core
Thread separada para rede + runOnUiThread	✓	Core
Layout portrait responsivo	✓	Core
Layout landscape responsivo	✓	Core
Temas Day/Night declarados (4 temas)	✓	Core
Ícone meteorológico por WMO code	✓	Core
Strings PT/EN (i18n)	✓	Core
Permissão INTERNET no Manifest	✓	Core
Ícone da aplicação personalizado	✓	Core
Background landscape diferente do portrait	⚠	Core
Layout tablet (sw600dp)	✗	Core
Fetch ao arrancar (dados visíveis sem premir UPDATE)	✗	Core

day calculado por sunrise/sunset da API	X	Core
GPS + recursos XML (challenge 2.2.1)	X	+5%
MVVM com LiveData (challenge 2.2.2)	X	+5%

7. Conclusão

O projeto da Cool Weather App demonstra uma implementação sólida das funcionalidades core exigidas. A integração com a API Open-Meteo está bem executada, com boas práticas de threading, tratamento de erros e deserialização JSON. Os layouts portrait e landscape para telemóvel estão implementados de forma adequada, os temas Day/Night estão declarados e a internacionalização PT/EN está corretamente configurada.

As principais lacunas a corrigir antes da entrega são: a ausência do fetch inicial no onCreate() (que deixa a app sem dados ao arrancar), o layout tablet em falta, e a determinação dinâmica do valor de day com base nos dados reais de sunrise/sunset. Estas correções são relativamente simples de implementar e aumentariam significativamente a qualidade e completude do projeto.

No geral, a base de código está bem estruturada e o projeto está próximo de cumprir todos os requisitos core do tutorial.

⚠ Nota: Esta secção do tutorial é marcada com [AC NO, AI NO]. Todo o código deve ser desenvolvido de forma autónoma, sem recurso a ferramentas de IA.